

K-Means Clustering — Lab Assignment

Goal. Implement K-Means clustering from scratch and understand how the algorithm partitions unlabeled data into K clusters by minimizing intra-cluster variance. You will compute the objective, visualize convergence and cluster assignments, and automatically identify the optimal number of clusters via validation-based criteria. (No scikit-learn for the K-Means training itself.)

Formulas

K-Means Objective

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

where $c^{(i)}$ is the index of the centroid assigned to $x^{(i)}$.

Centroid Update

$$\mu_k = \frac{\sum_{i=1}^m \mathbf{1}\{c^{(i)} = k\} x^{(i)}}{\sum_{i=1}^m \mathbf{1}\{c^{(i)} = k\}}$$

Within-Cluster Sum of Squares (WCSS)

$$\text{WCSS}(K) = \sum_{k=1}^K \sum_{x^{(i)} \in C_k} \|x^{(i)} - \mu_k\|^2$$

Silhouette Coefficient

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad a(i) = \text{mean distance to points in same cluster}, \quad b(i) = \min_{\ell \neq c^{(i)}} \text{mean distance to cluster } \ell$$

Exercise 2 — Automatic Selection of Optimal K (Elbow Method Only)

Objective. To automatically determine the optimal number of clusters K in K-Means clustering using the *Elbow Method*, evaluated on a validation set.

Task.

(a) For each $K \in \{1, 2, \dots, K_{\max}\}$:

- Train the K-Means algorithm on the `train.csv` data using *k-means++* initialization.
- Assign the validation data (`val.csv`) to the nearest centroids and compute the **Within-Cluster Sum of Squares (WCSS)**:

$$\text{WCSS}(K) = \sum_{i=1}^{m_{\text{val}}} \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

where $\mu_{c^{(i)}}$ denotes the centroid of the cluster assigned to $x^{(i)}$.

- (b) Determine the **Elbow Point** K_{elbow} using the *kneedle* rule: Let the points $(K, \text{WCSS}(K))$ for $K = 1, \dots, K_{\text{max}}$ define a curve. The elbow is identified as the K having the maximum perpendicular distance from the straight line connecting $(1, \text{WCSS}(1))$ and $(K_{\text{max}}, \text{WCSS}(K_{\text{max}}))$:

$$K_{\text{elbow}} = \arg \max_K d_K$$

where

$$d_K = \frac{|(K_{\text{max}} - 1)(\text{WCSS}_1 - \text{WCSS}_K) - (1 - K)(\text{WCSS}_1 - \text{WCSS}_{K_{\text{max}}})|}{\sqrt{(K_{\text{max}} - 1)^2 + (\text{WCSS}_1 - \text{WCSS}_{K_{\text{max}}})^2}}.$$

- (c) Retrain K-Means on the training data using $K = K_{\text{elbow}}$ and evaluate on the test data (`test.csv`). Compute and report:

$$J_{\text{test}} = \frac{1}{m_{\text{test}}} \sum_{i=1}^{m_{\text{test}}} \|x^{(i)} - \mu_{c(i)}\|^2, \quad \text{WCSS}_{\text{test}} = \sum_{i=1}^{m_{\text{test}}} \|x^{(i)} - \mu_{c(i)}\|^2.$$

Input Format.

- Two lines are provided via standard input:

```
Kmax=<int>
iterations=<int>
```

- Three files are present in the working directory:

- `train.csv` — training data
- `val.csv` — validation data
- `test.csv` — test data

Output Format. The program must print exactly three lines to standard output (values rounded to two decimals):

```
Elbow_K=<int>
WCSS_Val=[W1,W2,...,WKmax]
Test_J_Elbow=<float> Test_WCSS_Elbow=<float>
```

where

- Elbow_K is the optimal K_{elbow} .
- WCSS_Val is the validation WCSS list for $K = 1, \dots, K_{\text{max}}$.
- Test_J_Elbow and Test_WCSS_Elbow are computed on the test set.

Constraints.

- $1 \leq K_{\text{max}} \leq \min(12, m_{\text{train}})$.
- `iterations` $\in [1, 100]$.
- Only `numpy` and `matplotlib` may be used. Do not import or call `sklearn.cluster.KMeans`.

Expected Behavior.

- (i) Compute $\text{WCSS}(K)$ on validation data for all K .

- (ii) Select K_{elbow} via the kneedle rule.
- (iii) Retrain and compute J_{test} and $\text{WCSS}_{\text{test}}$.
- (iv) Save the optional plot `elbow_curve.png`.

Sample Input.

```
Kmax=6  
iterations=10
```

Sample Output.

```
Elbow_K=3  
WCSS_Val=[1379.26,638.25,106.82,94.85,80.87,70.47]  
Test_J_Elbow=1.24  Test_WCSS_Elbow=111.57
```

Evaluation Criteria (for VPL/Moodle).

- Correct computation of $\text{WCSS}(K)$ values.
- Proper identification of K_{elbow} .
- Accuracy of J_{test} and $\text{WCSS}_{\text{test}}$ within a small numeric tolerance (± 0.05).
- Output format matches exactly as specified.

Implementation Hints.

- Use *k-means++* initialization with a fixed random seed (e.g., 0) for reproducibility.
- Handle empty clusters by reseeding the centroid with a random training point.
- Ensure convergence detection based on unchanged cluster assignments.
- When computing squared distances, clip small negative values to 0 to avoid numeric errors.